

웨딩올인원 AI

1. 커뮤니티 AI 한줄 요약 : 게시글을 작성하면 AI가 본문을 읽고 한 문장으로 요약해서 목록에 바로 보여준다.
2. AI 웨딩 플랜 : 예산·지역·취향을 말하면 AI가 홀/스튜디오/드레스/메이크업 조합을 추천해준다.
3. AI 챗봇 : 궁금한 걸 물어보면 AI가 필요한 기능(상품 검색 등)을 스스로 판단해서 호출한 뒤 답변해준다.
4. AI 드레스 : 가상피팅용 AI에서 드레스를 입고, 프롬프트를 입력하면 GPT가 인식을 해 배경을 만들어준다.
5. AI 견적 분석 : 종이 견적서를 업로드하면 AI가 글자를 읽어서 항목별로 정리해주고, 같은 종류 견적서끼리는 가격·조건 비교도 해준다.
6. AI 운영관제 : 매일 새벽 사이트에 이상이 없는지 AI가 자동으로 점검하고, 매주 한 번은 운영 현황을 정리한 브리핑을 만들어준다.

팀장 황용현
팀원 이재원 · 권용익 · 윤승진

Team GitHub:

https://github.com/sai0734/Sixfour_Team

Team YouTube:

https://youtu.be/ltr_3dvikuc?si=horohbVLppz1mMK-

AI 도입목적

왜 웨딩올인원에 AI가 필요한가 - 선택 피로를 줄이고, 체험·검증·운영까지 확장

문제 1

정보 과다 · 선택 피로

홀·스드메·예산·일정이 사이트마다 흩어져
비교 기준이 없고 결정이 계속 미뤄짐

문제 2

반복 상담 · 견적 비교

여러 업체에 같은 질문을 반복하고
견적 조건이 제각각이라 합리적 비교가 어려움

문제 3

사진 기반 체험의 한계

카탈로그·후기 사진만으로는
나에게 어울리는지를 미리 확인하기 어려움

AI가 담당하는 역할

요약

커뮤니티
한줄요약

추천

AI
웨딩플랜

상담

AI 챗봇

체험

AI 드레스
가상피팅

검증

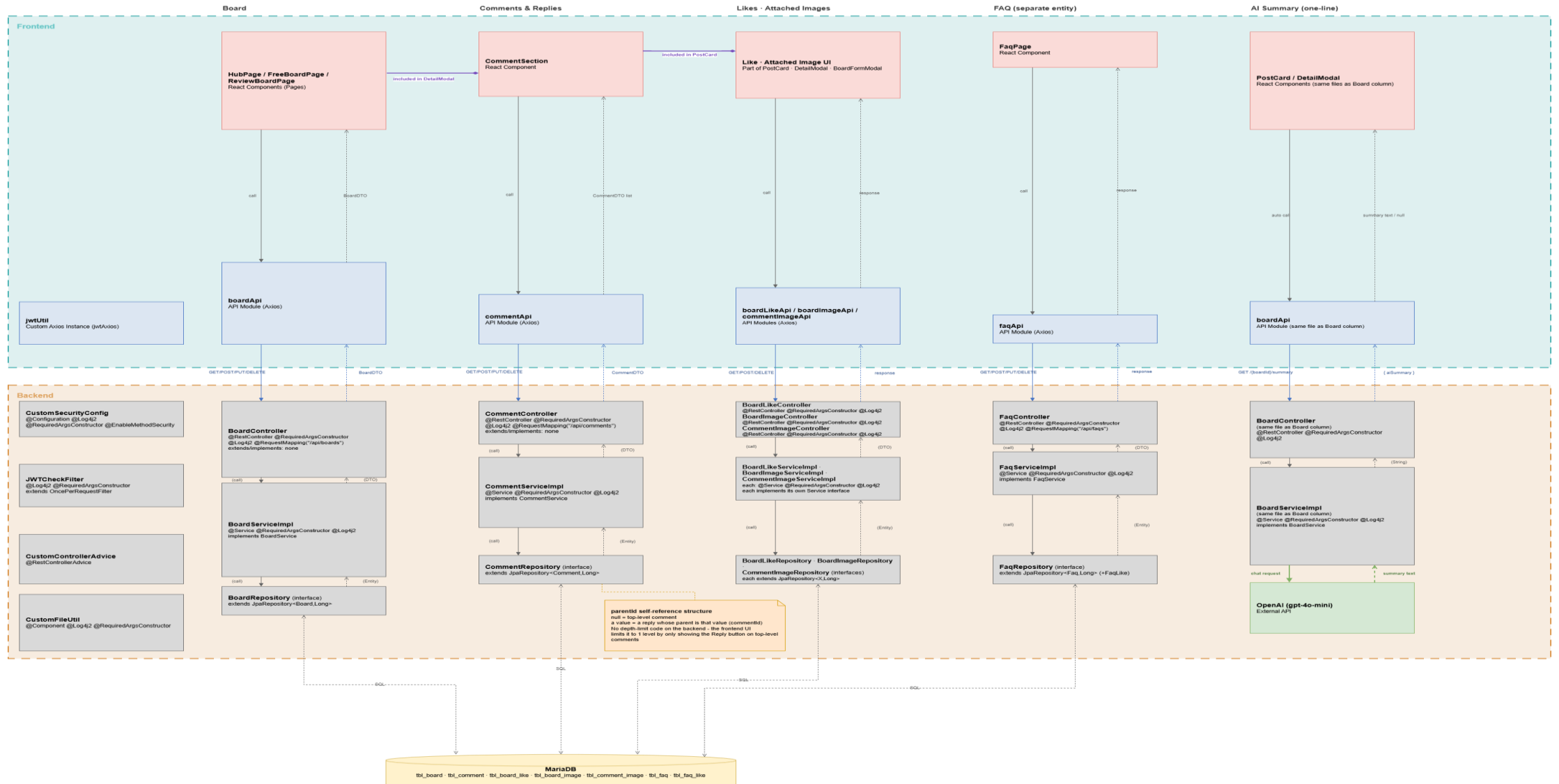
AI 견적서

운영

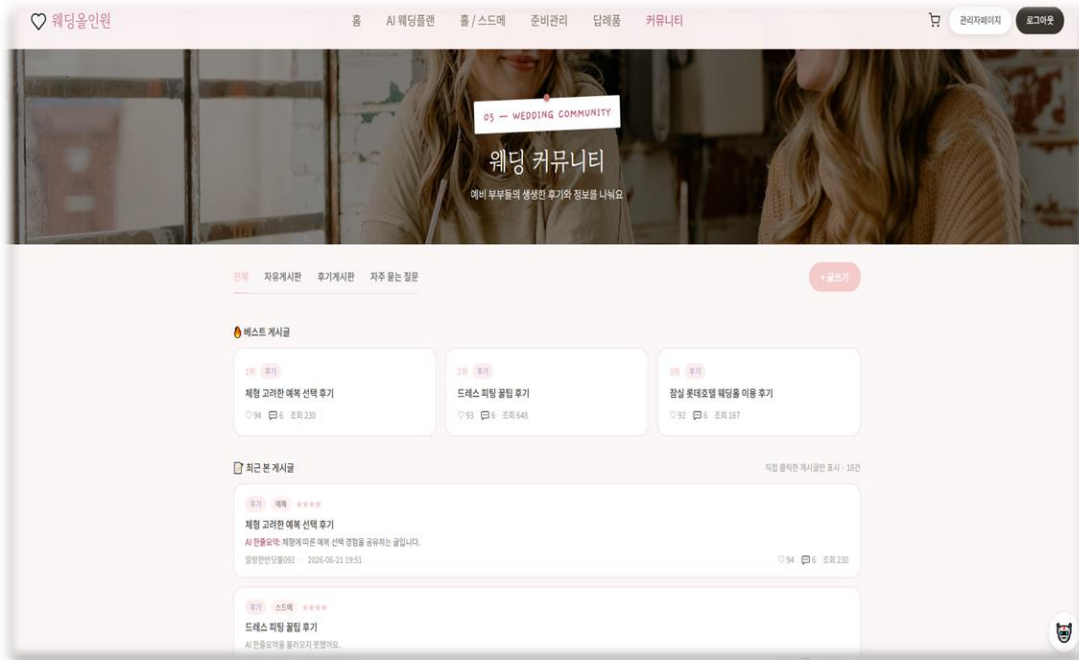
OpenClaw
운영관제

커뮤니티 및 AI 한줄요약 Flow

Spring Data JPA + OpenAI Chat Completions API



커뮤니티 및 AI 한줄요약



```
const topLevel = comments.filter((c) => !c.parentId); if (!comment.isDeleted()) {  
const repliesOf = (parentId) =>                                comment.changeDeleted(true);  
    comments.filter((c) => c.parentId === parentId);                commentRepository.save(comment);  
}
```

서버는 댓글을 시간순 평평한 목록으로만 내려주고, parentId 유무로 최상위 댓글/대댓글을 나눠 프론트에서 트리 구조로 렌더링 - 부모 댓글이 삭제돼도 소프트 삭제 처리라 대댓글은 그대로 살아있습니다.

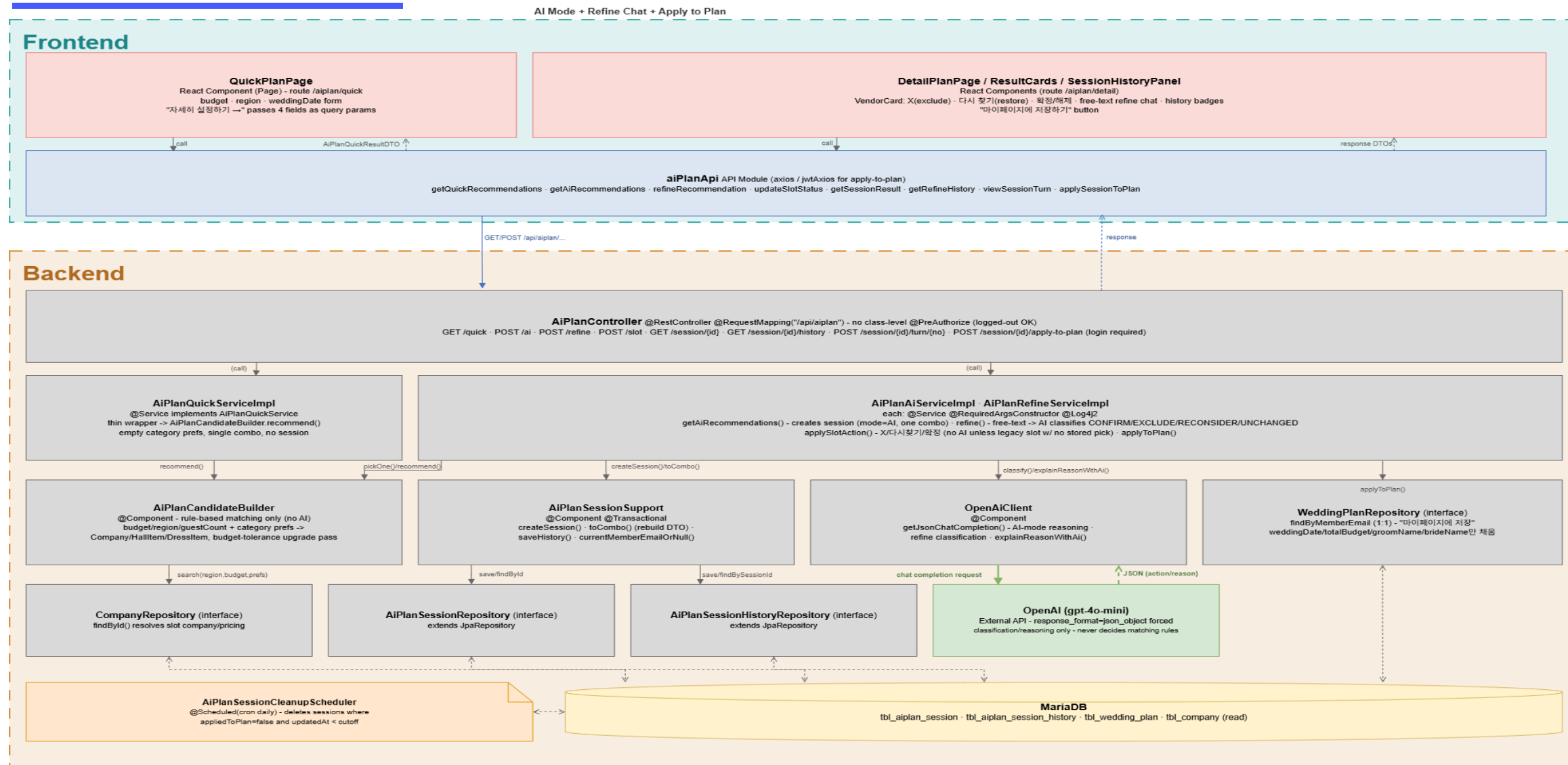
```
if (board.getAiSummary() != null && !board.getAiSummary().isBlank()) {  
    return board.getAiSummary(); // 이미 있으면 OpenAI 호출 없이 바로 반환  
}  
  
OpenAiResponseDTO response = openAiClient.getChatCompletions(messages, tools: null);  
String summary =  
    response.getChoices().getFirst().getMessage().getContent().trim();  
  
board.changeAiSummary(summary); // DB에 캐싱  
boardRepository.save(board);
```

게시글마다 딱 한 번만 OpenAI를 호출하고 결과를 Board.AISummary 컬럼에 저장 -> 이후 조회는 DB 캐시로 응답, 비용.응답속도 문제를 함께 해결. 글 수정 시엔 changeAISummary(null)로 캐시를 비워 다음 조회 때 재 생성되게 처리합니다.

```
useEffect(() => {  
    if (!isSummaryEligible || post.aiSummary || post.boardId == null) return;  
    // 캐시 있거나 짧으면 호출 안 함  
    getAiSummary(post.boardId).then(...);  
}, [post.boardId, post.aiSummary, isSummaryEligible]);
```

본문 150자 미만인 글은 요약이 별 의미가 없다고 판단해 API 요청 자체를 생략, 목록 응답에 이미 캐시된 요약이 있으면 그것도 재요청하지 않음 -> 불필요한 OpenAI호출을 프론트 단에서부터 줄입니다.

AI 웨딩플랜 Flow



AI 웨딩플랜



드레스

드레스 · 에브와르 청담
벨슬리브 드레스
439,000원

에브와르 청담의 드레스는 A라인과 오프숄더 디자인으로 섬세하게 만들어져 있어, 우아함과 세련됨을 강조하며 신부의 아름다움을 극대화합니다.

이 업체로 확정하기



메이크업

메이크업 · 아틀리에 웨딩
풀 패키지
734,800원

아틀리에 웨딩의 FULL 패키지는 하객이 많은 특별한 날 완벽한 헤어와 메이크업을 제공하여 전체적인 스타일에 고급스러움을 더해줄 것입니다.

이 업체로 확정하기

예식일 · 총예산이 마이페이지 플랜에 반영돼요

마음에 안 드는 부분이 있나요?
자유롭게 말씀해주시면 그 부분만 다시 찾아드려요

답기

스튜디오는 빼줘 드레스 다른 스타일로 다시 찾아줘 홀은 마음에 들어서 확정할게 예산 좀 더 늘려서 다시 찾아줘

예: 스튜디오는 예산 초과라서 빼고 나머지로 추천해줘. 홀이랑 메이크업은 마음에 들어서 확정, 드레스만 다른 스타일로 다시 찾아줘

보내기

다른 조건으로 새로 찾기 링크로 공유하기

```
@Scheduled(cron = ""0 0 0 * * *") @Transactional
public void cleanupUnappliedSessions() {

    LocalDateTime cutoff = LocalDateTime.now().minusDays(sessionTtlDays);
    List<AiPlanSession> expired =
        sessionRepository.findByAppliedToPlanFalseAndUpdatedAtBefore(cutoff);
```

답기(플랜 적용)까지 이어지지 않고 방치된 세션만 골라 일정 기간 지나면 매일 자정 자동 삭제 — 실제 사용 중인 세션은 건드리지 않습니다.

```
case "EXCLUDE" -> SlotStatus.EXCLUDED;
case "RECONSIDER" -> SlotStatus.PENDING;
default -> null; // UNCHANGED - 상태를 안 바꿈
```

사용자의 자연어 요청을 AI가 JSON으로 강제 응답시켜 카테고리별 CONFIRM/EXCLUDE/RECONSIDER로 분류 — 파싱 실패 시 조용히 규칙 기반으로 넘어가지 않고 세션을 그대로 유지합니다.

```
long leftover = budget - total;

for (CompanyCategory category : UPGRADE_ORDER) {
    if (leftover <= BUDGET_TOLERANCE) {
        break;
    }

    Pick current = picks.get(category);
    if (hasPreference(category, prefs) && !current.usedStyleFallback) {
        continue; // 취향에 맞춰 고른 곳은 예산 때문에 임의로 안 바꿈
    }
}
```

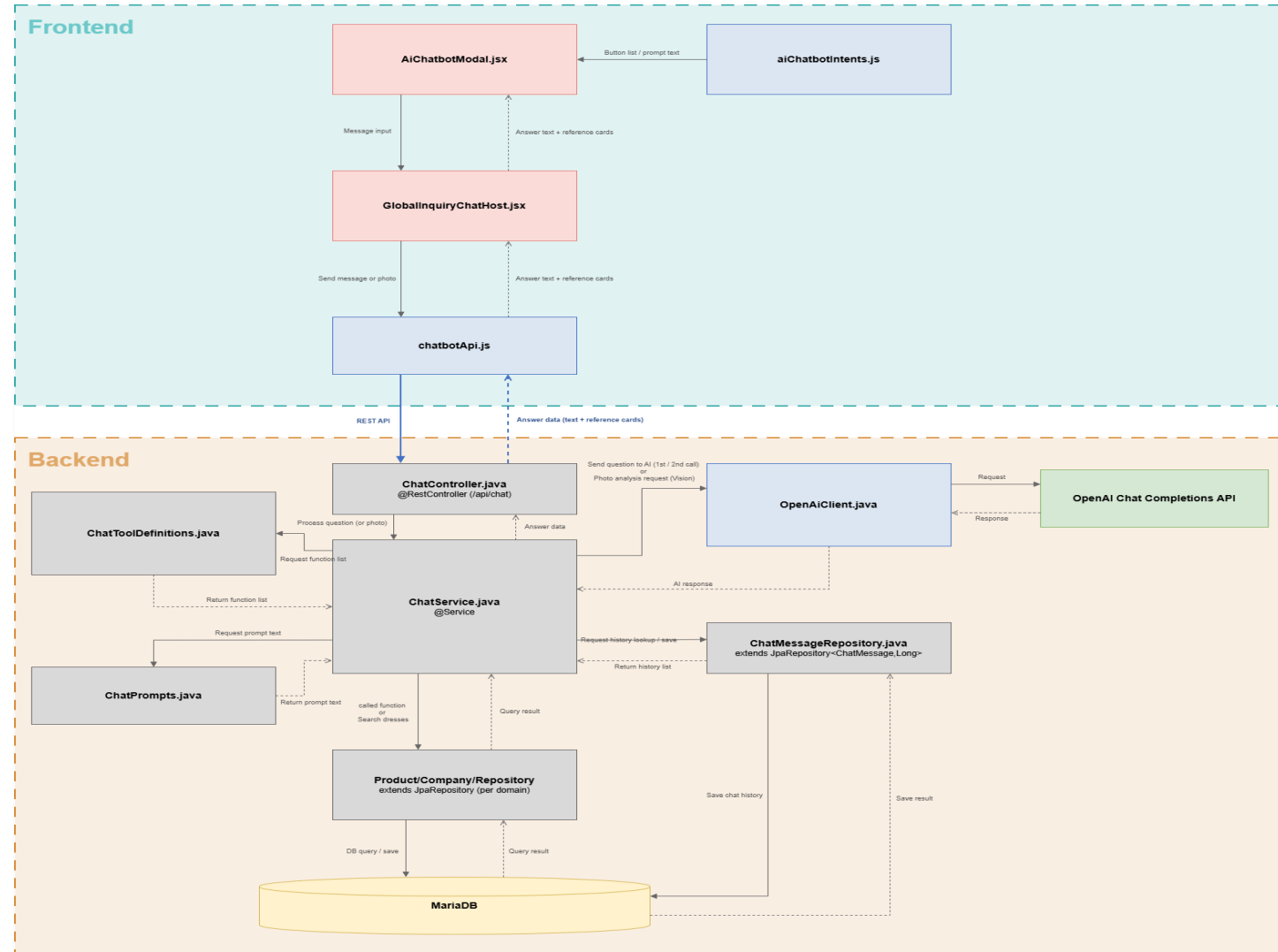
1차 배분 후 남은 예산이 있으면 취향이 반영 안 된 카테고리부터 더 비싼 옵션으로 업그레이드 — 사용자가 명확히 취향을 지정한 카테고리는 예산 때문에 임의로 안 바뀝니다.

AI 챗봇 Flow

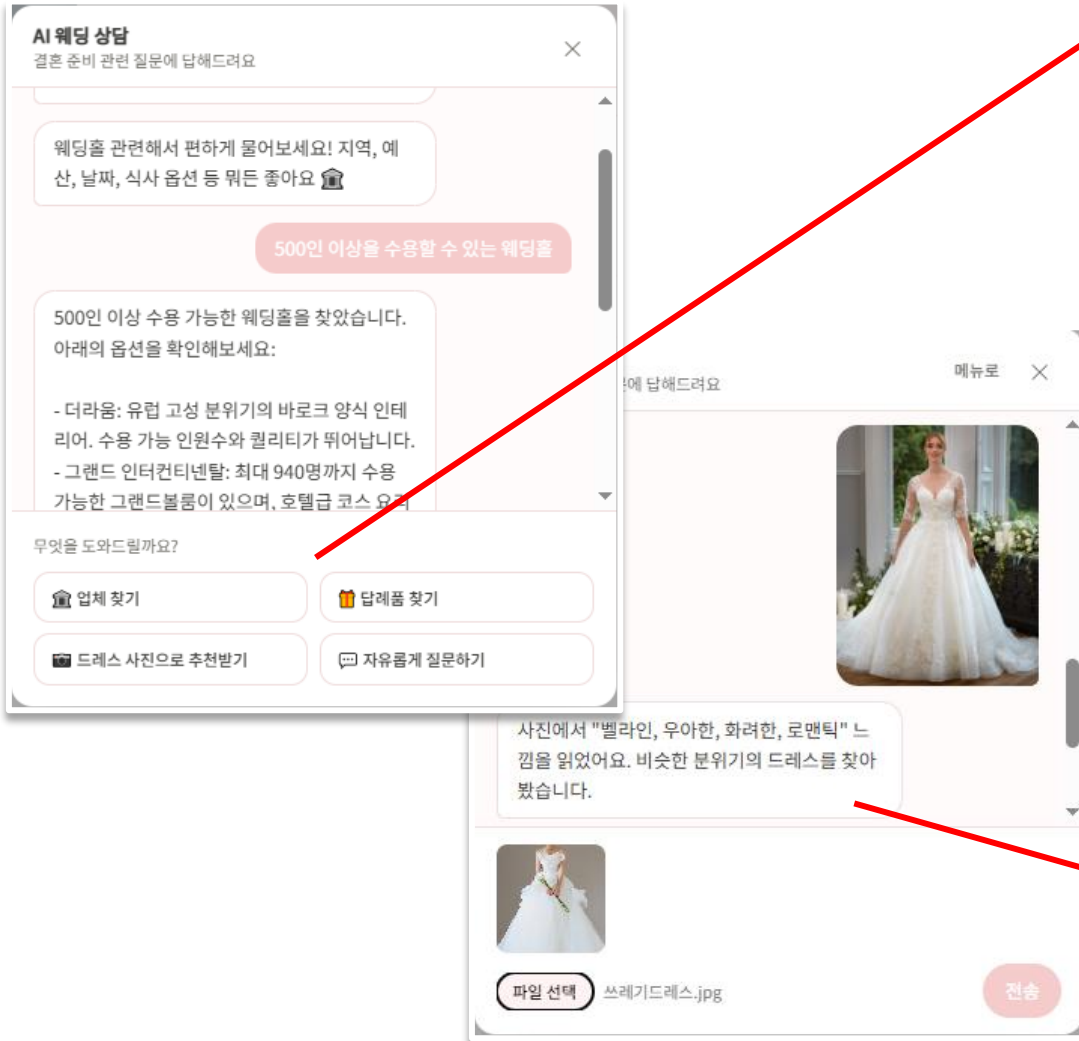
OpenAI Function/Tool



- 프론트엔드와 Spring Boot 백엔드, OpenAI API를 연동한 챗봇 시스템입니다.
- 사용자의 질문에 지능적인 답변을 할 수 있도록 프롬프트와 로직을 체계화했습니다.
- 대화가 끊기거나 사라지지 않도록 DB에 안전하게 저장합니다.



AI 챗봇



```
OpenAiResponseDTO firstResponse = openAiClient.getChatCompletions(  
    messages, ChatToolDefinitions.resolveTools(intent), toolChoice: "required");  
1차 조회  
OpenAiMessageDTO assistantMessage = firstResponse.getChoices().getFirst().getMessage();  
String finalAnswer;  
List<ChatReferenceDTO> references = new ArrayList<>();  
if (assistantMessage.getToolCalls() != null && !assistantMessage.getToolCalls().isEmpty()) {  
    List<OpenAiMessageDTO> followUp = new ArrayList<>(messages);  
    followUp.add(assistantMessage);  
    for (OpenAiMessageDTO.ToolCallDTO toolCall : assistantMessage.getToolCalls()) {  
        ToolResult result = executeToolCall(toolCall, lockedCategory);  
        references.addAll(result.references());  
        followUp.add(OpenAiMessageDTO.builder()  
            .role("tool")  
            .toolCallId(toolCall.getId())  
            .content(result.text())  
            .build());  
    }  
    2차 조회  
    OpenAiResponseDTO secondResponse = openAiClient.getChatCompletions(followUp, tools: null);  
    finalAnswer = secondResponse.getChoices().getFirst().getMessage().getContent();  
} else {  
    finalAnswer = assistantMessage.getContent();  
}
```

내가 정한 테두리에서
무조건 선택해라

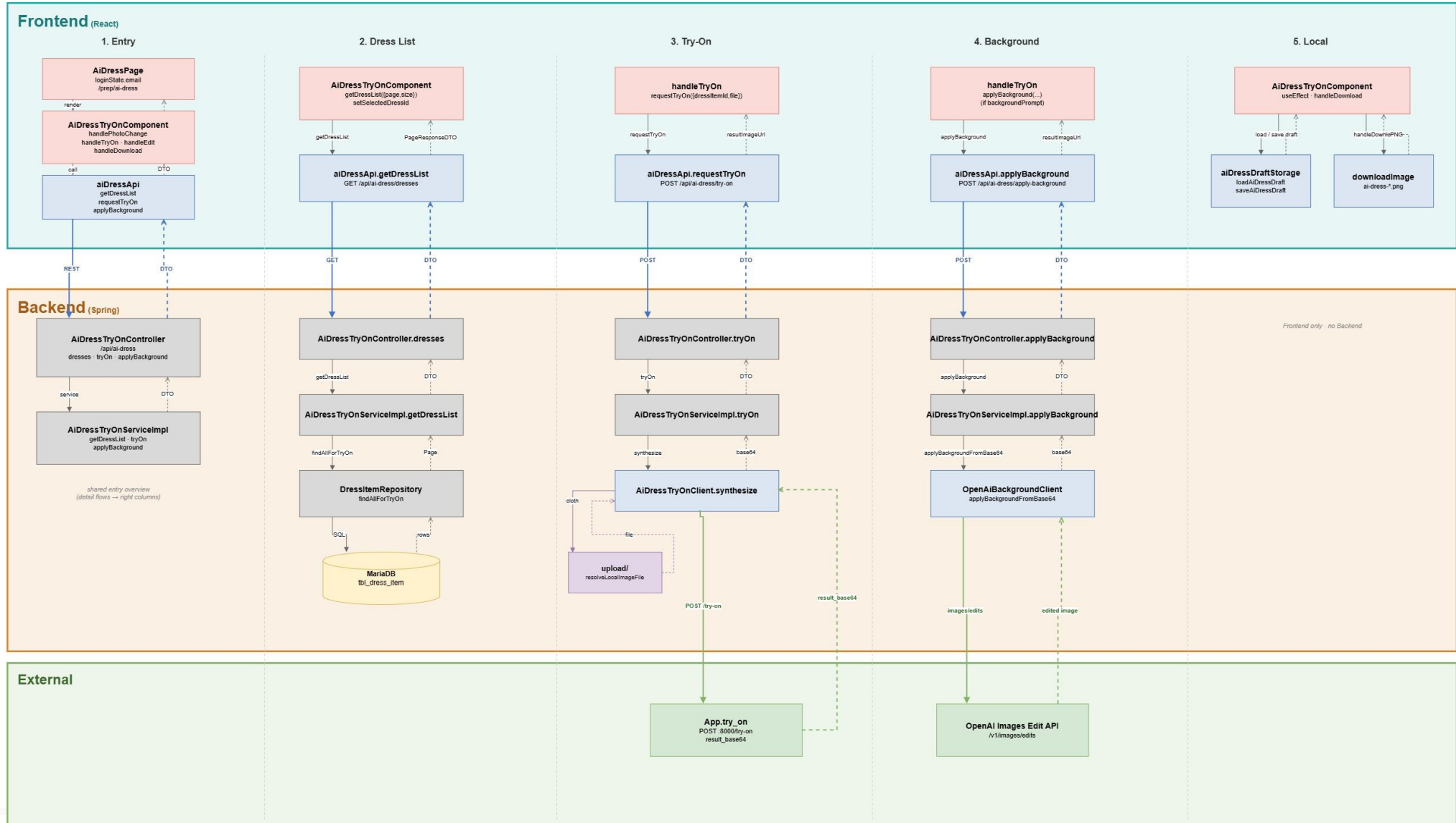
DB를 조회하여 실제
데이터를 뽑아온다

1단계로 intent(4가지 버튼)으로 좁혀진 함수들 중 required로 반드시 하나를 고르게 하고
2단계로 실제 DB를 조회한 결과를 다시 AI에게 보내 자연어로 답변을 받는 구조입니다.
덕분에 AI가 실제 정보를 기반으로 답변을 할 수 있습니다.

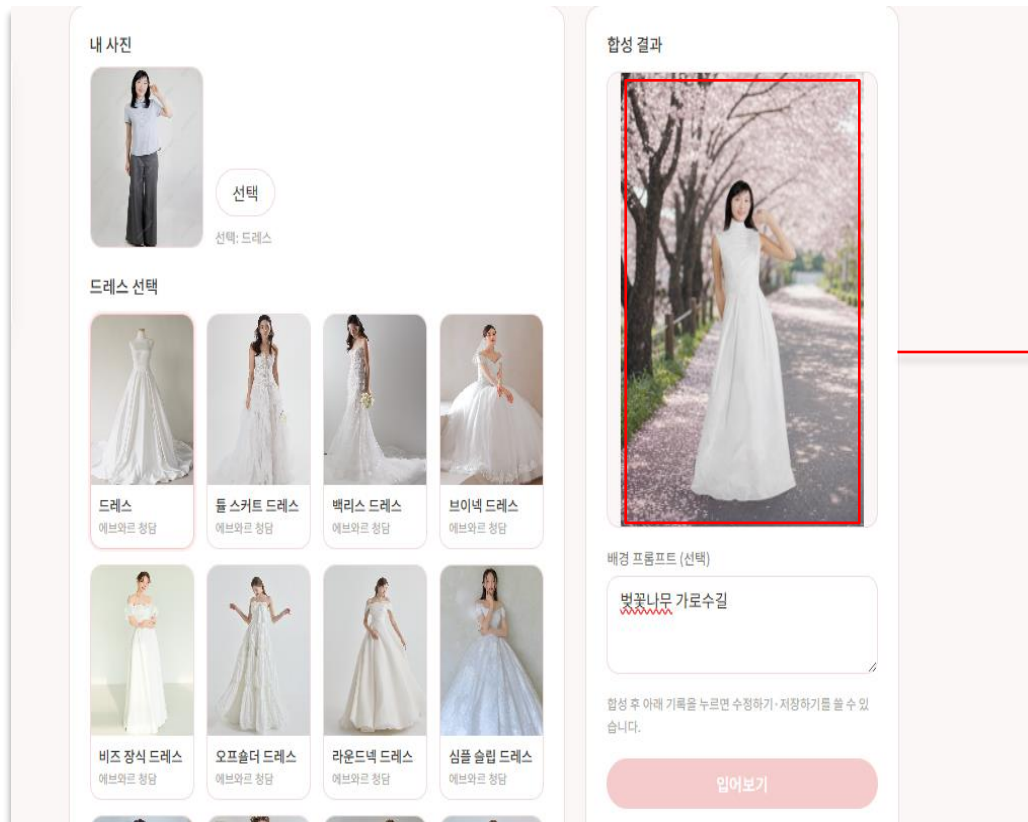
```
Map<Long, DressItem> itemsById = new LinkedHashMap<>();  
Map<Long, Integer> matchCount = new HashMap<>();  
for (String kw : keywordTerms) {  
    for (DressItem item : dressItemRepository.searchByStyleKeyword(kw, region: null, maxPrice: null)) {  
        itemsById.putIfAbsent(item.getDressItemId(), item);  
        matchCount.merge(item.getDressItemId(), value: 1, Integer::sum);  
    }  
}
```

추출된 키워드마다 DB를 검색하고 같은 드레스가 여러 키워드에 걸릴때마다 매칭 점수를
1점씩 더합니다. 겹치는 키워드가 많을수록 비슷한 드레스로 판단합니다.

AI 드레스 Flow 외부 CatVTON 이미지합성 API + OpenAI



AI 드레스

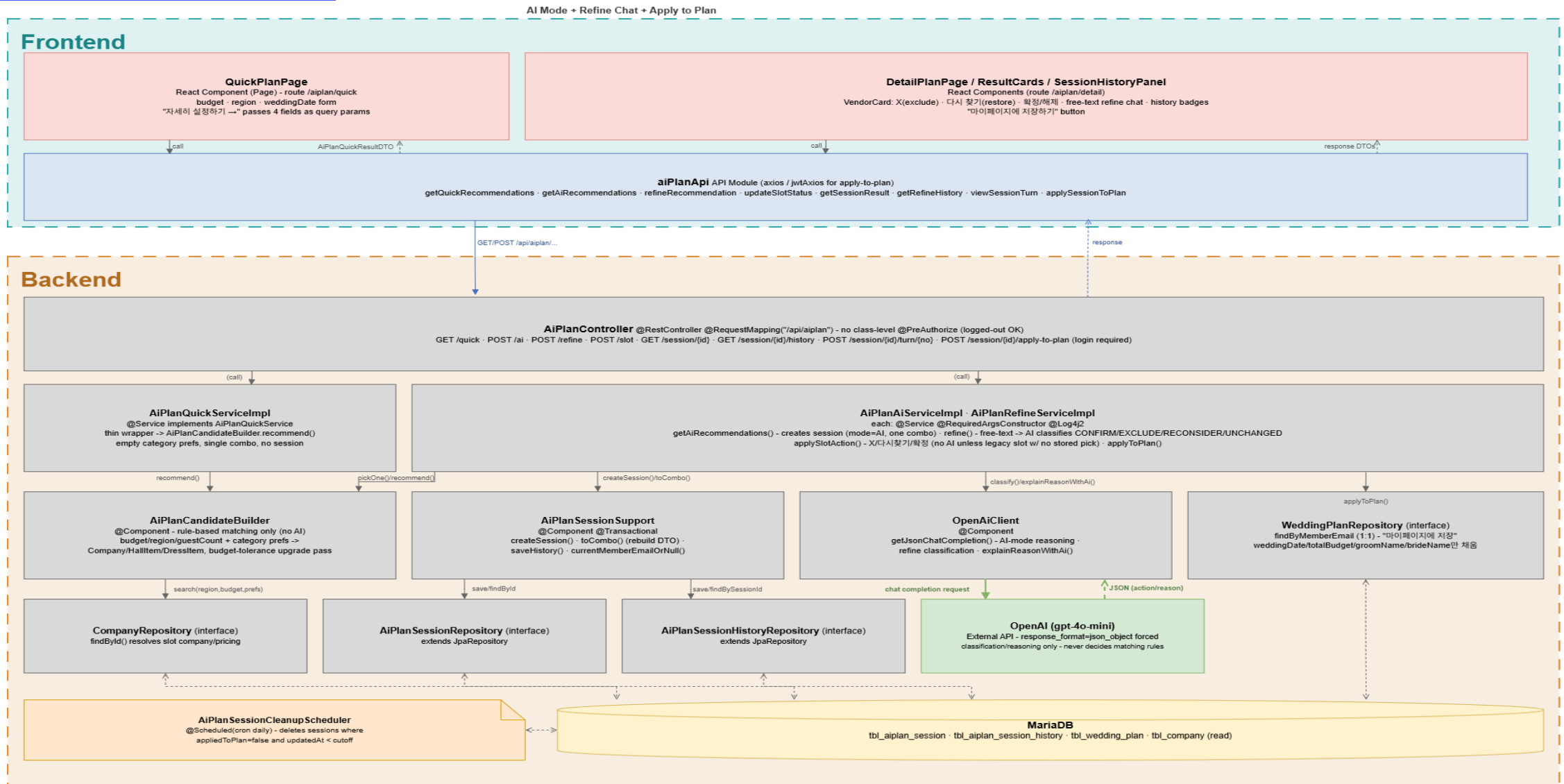


```
if mask is not None:
    mask_img = await load_image(mask, "L")
else:
    mask_img = make_mask(person_img)
try:
    results = pipeline(
        image=person_img,
        condition_image=cloth_img,
        mask=mask_img,
        num_inference_steps=50,
        guidance_scale=2.5,
    )
    buf = io.BytesIO()
    result_img.save(buf, format="PNG")
    b64 = base64.b64encode(buf.getvalue()).decode("utf-8")
    return JsonResponse({"result_base64": b64})
```

Spring이 사람사진과 드레스 이미지를 / try-on으로 보내면, 마스크를 준비한 뒤 파이프라인으로 가상피팅을 돌리고, 결과PNG를 Base64로만 반환합니다. 프로젝트 저장소에는 이 API서버 코드가 있고, 실제 모델로드는 GPU 환경에서 pipeline을 초기화해서 사용합니다.

AI 견적서 Flow

Google Vision OCR + OpenAI Chat Completions(JSON)



AI 견적서

💰 비교 결과

가격 차이

A가 B보다 45,774,000원 저렴합니다.

조건 차이

· A업체는 잔금이 현금가로 1,000,000원이며, 부가세는 별도로 포함되어 있지 않고 봉사료는 포함되어 있다.
· B업체는 예식 용품이 포함되며, 서비스 봉사료와 부가세가 각각 10% 포함되어 있다.

웨딩홀 · (주)아트리블름

총액: 8,200,000원	
대관료 (포함)	3,000,000원
프리미엄 디렉팅 (포함)	1,500,000원
발렛주차 (포함)	1,200,000원
서둘차량 (포함)	500,000원
뷔페 (포함)	4,200,000원

- 부가세 별도, 봉사료 포함
- 잔금은 현금가로 1,000,000원

공통 특징

· 두 견적서 모두 총액이 명시되어 있으며, 특정 항목의 가격이 포함되어 있다.

확인해볼 질문

· A업체에 물어볼 것: 부가세는 별도로 계산되는지, 잔금 유무 및 결제 방식에 대해 더 상세한 정보를 요청하기.
· B업체에 물어볼 것: 예식 용품의 구체적인 내용을 확인하고, 부가세와 봉사료가 포함된 가격에 대해 문의하기.

웨딩홀 · 업체명 미상

총액: 53,974,000원	
식사 (포함)	27,000,000원
Welcome Drink (포함)	1,620,000원
Champagne (포함)	150,000원
꽃장식 (포함)	13,000,000원

- 예식 용품 포함
- 10% 서비스 봉사료 및 10% 부가세 포함

견적서 업로드

웨딩홀/스튜디오/드레스/메이크업 견적서 사진(JPG/PNG)을 올리면 AI가 항목을 정리해요. 업로드 후 30일이 지나면 자동으로 삭제돼요.
손글씨로 등그림이·메모가 적혀 있거나 사진이 흐릿하면 정확도가 떨어질 수 있어요. 가능하면 깨끗하고 선명한 사진을 올려주세요.

사진 선택

이 사진은 웨딩 업체 견적서가 아니라 소프트웨어 개발 견적서로 보입니다.

QuoteServiceImpl.java

AI 판단을 서버가 검증·실행하는 코드 — 실제 프롬프트 원문은 파일 내 별도 정의

```
// 카테고리 강제의 실질적 지점 - 홀 견적서와 드레스 견적서처럼 서로 다른 종류는 비교 자체를 막는다.  
if (a.getCategory() != b.getCategory()) {  
    throw new IllegalArgumentException("같은 종류의 견적서끼리만 비교할 수 있어요.");  
}
```

홀-홀, 드레스-드레스처럼 같은 종류끼리만 비교를 허용합니다 — AI가 서로 다른 종류의 견적서를 놓고 "어느 쪽이 낫다"는 억지 비교를 하지 않도록 원천 차단합니다

```
// 가격 차이는 AI 문장이 아니라 저장된 totalPrice로 서버가 직접 계산 - 큰 숫자에 콤마  
// 구분을 빠뜨리는 등 AI의 숫자 포매팅 실수를 원천적으로 없앤다.  
String priceDifference = buildPriceDifference(dtoA, dtoB);
```

가격 차이 문구는 AI가 생성한 텍스트가 아니라 저장된 금액으로 서버가 직접 계산 — AI가 숫자를 잘못 세거나 콤마를 빠뜨리는 실수를 원천 차단합니다.

```
if (rejectReason != null && !rejectReason.isBlank()) {  
    throw new IllegalStateException(rejectReason);  
}  
  
if (category == null) {  
    throw new IllegalStateException("어떤 종류의 견적서인지 확인하지 못했어요. 홀/스튜디오/드레스/메이크업 견적서만 지원해요.");  
}
```

AI가 웨딩 업체 견적서가 아니라고 판단하거나(자동차·가전 등) 카테고리를 확신 못하면 업로드 자체를 거절하고, 구체적인 거절 사유를 그대로 사용자에게 보여줍니다.

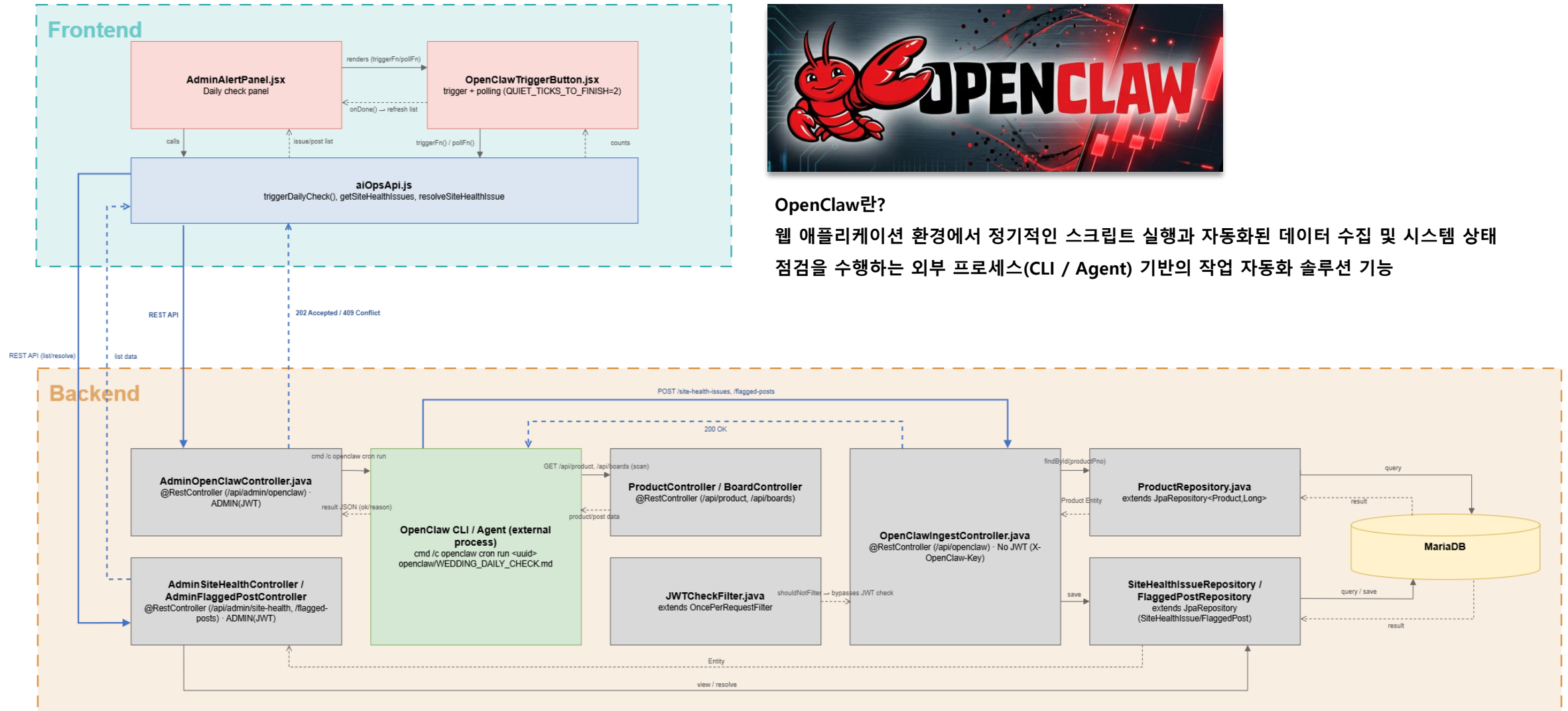
AI 운영관제 Flow

OpenClaw(로컬 자동화 에이전트) + 스케줄 배치 + 브라우저 자동조작



OpenClaw란?

웹 애플리케이션 환경에서 정기적인 스크립트 실행과 자동화된 데이터 수집 및 시스템 상태 점검을 수행하는 외부 프로세스(CLI / Agent) 기반의 작업 자동화 솔루션 기능



AI 운영관제

일일체크

4건

사이트 이상 징후

이미지 깨짐

http://localhost:3000/product/list

답례품 카테고리 7번째 상품에 실제 사진이 없어 기본 이미지가 대신 표시되고 있습니다.

처리 완료

확인 필요한 게시물

저격성

게시글 #91

특정 회원을 반복적으로 저격하는 듯한 표현이 있어 확인이 필요합니다.

처리 완료

욕설

게시글 #101

다른 회원을 향한 명확한 욕설/비하 표현이 포함되어 있어 확인이 필요합니다.

처리 완료

지금 바로 진단하기

```
String outputText = runCommand("cron", "run", jobId);

Boolean ok = null;
JsonObject json = gson.fromJson(outputText.trim(), JsonObject.class);
if (json != null && json.has("ok")) {
    ok = json.get("ok").getAsBoolean();
}

if (Boolean.TRUE.equals(ok)) {
    return ResponseEntity.accepted().body(outputText);
}
return ResponseEntity.status(HttpStatus.CONFLICT)...;
```

실제로 성공 / 실패를
판단하는 로직

트리거 역할로 관리자가 대시보드의 버튼을 클릭하거나 매일 새벽 3시가 되면 로컬에 설치된 OpenClaw CLI가 실행되어 일일체크를 실행합니다.

작업은 이름이 아닌 고유 ID로만 실행할 수 있기 때문에 실행 전에 이름 -> ID를 조회하는 과정을 거치도록 구현했습니다.

```
if (!p.uploadFileNames || p.uploadFileNames.length === 0) {
    issues.push({ pno: p.pno, pname: p.pname, type: "MISSING" });
    continue;
}
const check = await fetch(`.../api/product/view/${fileName}`, ...);
if (!check.ok) {
    issues.push({ pno: p.pno, pname: p.pname, type: "BROKEN" });
}
```

실제로 이미지가 열리는지 검증하는 로직

실행된 OpenClaw는 자체판단하여 백 엔드의 REST API를 호출하여 데이터를 조회 및 정상 로드가 되는지 직접 검증합니다. DB에 직접 접근하지 않고 백 엔드에 만들어둔 API를 통해서만 데이터를 확인하도록 설계했습니다.

```
String detail = dto.getDetail();
if ("IMAGE_BROKEN".equals(dto.getIssueType()) && dto.getProductPno() != null) {
    detail = productRepository.findById(dto.getProductPno())
        .map(p -> String.format("%s(상품번호 %d)의 대표 이미지를 불러올 수 없습니다.",
            p.getPname(), dto.getProductPno()))
        .orElseGet(...);
}

SiteHealthIssue issue = SiteHealthIssue.builder()...build();
siteHealthIssueRepository.save(issue);
```

AI가 판단한 제목이 아닌 정해진 제목을 구현

점검을 마친 OpenClaw는 발견한 문제를 콜백 API로 전송하고 서버는 이를 DB에 저장하여 관리자 화면에 즉시 반영합니다.

회고

주요 성과

- **AI 4종 통합** — 웨딩플랜·챗봇·드레스·견적서를 하나의 플랫폼에서 유기적으로 연결
- **실시간 커뮤니케이션 구축** — WebSocket(STOMP) 기반 업체-회원 실시간 문의 채팅 완성
- **성능 최적화 3종 적용** — N+1 방지, 서버 사이드 페이징, 코드 스플리팅으로 체감 속도 개선

배운 점

- **보안은 나중에 아니라 처음부터** — JWT 시크릿 하드코딩을 겪으며 설정값 외 부화 습관의 중요성 체감
- **"동작한다" ≠ "안전하다"** — 결제 승인 API는 여러 번 호출돼도 같은 결과가 나오게 처음부터 설계해야 함을 체득
- **초기 DB 설계는 완성본이 아니었다** — 개발 중 필드는 물론 테이블까지 새로 추가되며 유연한 스키마 설계의 중요성 체감

아쉬운 점

- **코드 컨벤션 미통일** — 팀원별 주석 스타일 차이로 리뷰 시 맥락 파악에 시간 소요
- **테스트 자동화 미흡** — 트러블슈팅 재현을 수동 캡처에 의존, 회귀 테스트 부재

향후 계획

- **로그인/회원가입** — 카카오 하나뿐인 소셜 로그인에 네이버·구글 추가, 휴대폰 본인인증(SMS) 연동으로 이메일 인증만으론 못 막는 허위가입까지 방지
- **AI견적서** — 2개 견적서 비교만 가능한 걸 3개 이상 동시 비교로 확장 항목별 시세 데이터와 비교해서 “이 항목이 평균보다 비쌉니다” 같은 인사이트 제공
- **AI운영관제** — 이상 감지 시 텔레그램/슬랙으로 실시간 알림 발송, 주간 브리핑에 전주 대비 추이그래프 추가

Thank you